

1. System Overview

Core Goal

Convert messy client input (text, voice notes, screenshots/files) into structured AI-generated project briefs that employees can review and share with clients through a secure read-only link.

Core Features Only

1. Intake system (upload + organization)
 2. AI structuring engine
 3. Shareable confirmation flow
-

2. High-Level Architecture

Recommended Stack

Frontend

- Framework: Next.js
- UI: Tailwind CSS
- State: React Query or Zustand

Backend

- API Service 1 (.NET): Intake + Brief Management
- API Service 2 (.NET): AI Processing + File Processing

Database

- PostgreSQL

Storage

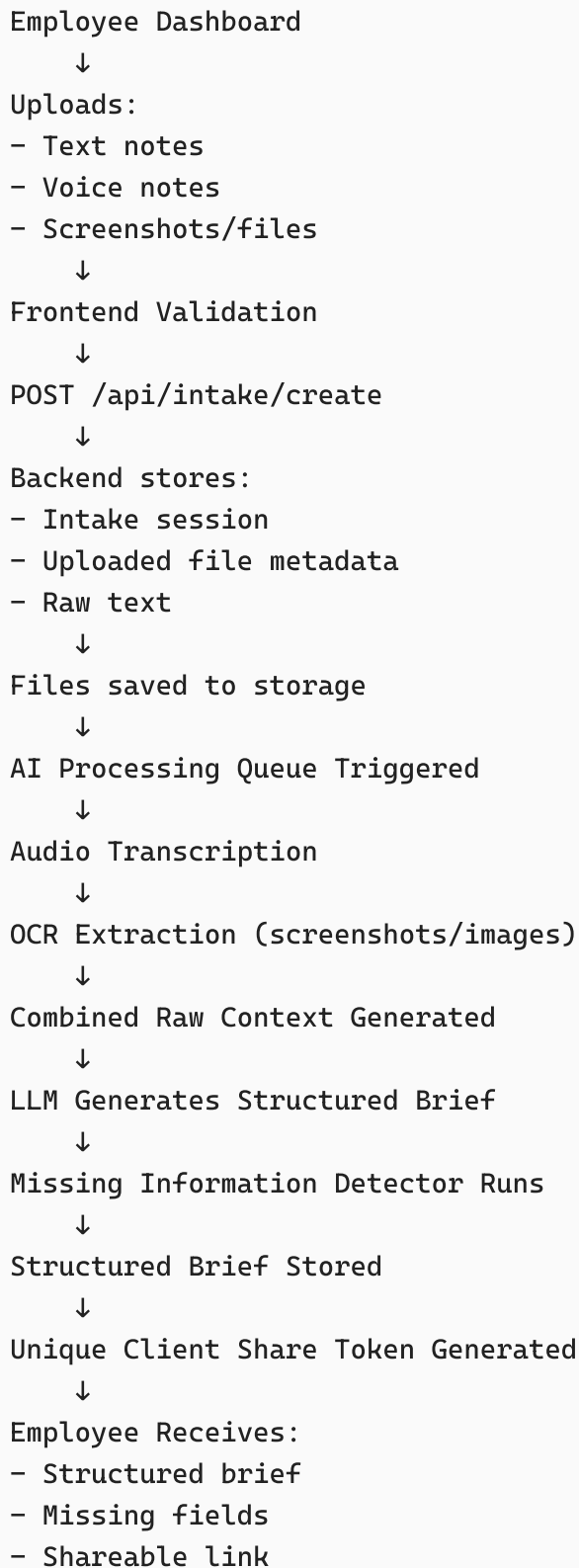
- Local file storage for hackathon simplicity
- Folder-based uploads

AI Layer

- Server-side LLM API wrapper

3. Complete Data Flow Diagram

A. Employee Upload Flow



B. Client Review Flow

```
Client Opens Shareable Link
↓
GET /api/public/brief/{token}
↓
Backend validates token
↓
Read-only brief returned
↓
Client reviews sections
↓
Client clicks:
- Confirm
OR
- Provide missing information
↓
POST /api/public/brief/{token}/answers
↓
Answers stored
↓
Employee notification flag created
↓
Employee dashboard updates
```

C. Employee Revision Flow

```
Employee opens dashboard
↓
GET /api/briefs
↓
Views:
- Pending confirmation
- Missing info requests
- Confirmed briefs
↓
Employee opens specific brief
↓
Client answers merged
↓
Employee edits/refines
↓
```

```
PATCH /api/briefs/{id}
```

↓

```
Database updated
```

4. Database Design

Core Tables

Users

```
users
- id
- name
- email
- password_hash
- created_at
```

Intake Sessions

```
intake_sessions
- id
- user_id
- title
- status
- created_at
```

Status

- uploaded
 - processing
 - generated
 - awaiting_client
 - confirmed
-

Uploaded Files

```
uploaded_files
- id
- intake_session_id
- file_name
- file_type
- storage_path
- extracted_text
- created_at
```

Briefs

```
briefs
- id
- intake_session_id
- structured_json
- markdown_output
- missing_fields_json
- ai_summary
- status
- created_at
- updated_at
```

Public Share Links

```
share_links
- id
- brief_id
- token
- expires_at
- created_at
```

Client Responses

```
client_responses
- id
- brief_id
```

- field_name
 - response_text
 - created_at
-

5. Structured Brief Format

Recommended JSON Structure

```
{
  "project_name": "",
  "client_name": "",
  "business_goal": "",
  "target_users": [],
  "features": [],
  "platforms": [],
  "design_requirements": [],
  "technical_constraints": [],
  "timeline": "",
  "budget": "",
  "missing_information": []
}
```

6. Core Use Cases

Use Case 1 — Employee Uploads Messy Data

Actor

Employee

Flow

1. Employee creates intake session
2. Uploads files and notes
3. Backend stores uploads
4. AI extracts content
5. AI generates structured brief
6. Missing fields identified

7. Employee reviews generated brief

Output

Structured project brief + shareable link

Use Case 2 — Missing Information Detection

Actor

System

Flow

1. AI evaluates completeness
2. Required fields checked
3. Missing items stored
4. Employee shown missing items
5. Client optionally asked through share link

Output

List of unresolved information gaps

Use Case 3 — Client Review & Confirmation

Actor

Client

Flow

1. Client opens secure link
2. Reviews generated brief
3. Adds missing information if requested
4. Confirms brief

Output

Confirmed project brief

Use Case 4 — Employee History Access

Actor

Employee

Flow

1. Employee logs in
2. Opens dashboard
3. Views past briefs
4. Reopens/edit/shares existing brief

Output

Persistent searchable history

7. Backend Service Separation

Backend Service 1 — Core API

Responsible for:

- Authentication
- Intake sessions
- Brief CRUD
- Public share links
- Client responses

Backend Service 2 — AI Processing

Responsible for:

- File processing
 - OCR
 - Audio transcription
 - LLM prompting
 - Missing-field analysis
-

8. API Design

Authentication

POST /api/auth/login

Request

```
{  
  "email": "",  
  "password": ""  
}
```

Response

```
{  
  "token": "",  
  "user": {}  
}
```

Intake APIs

POST /api/intake/create

Multipart Form Data

```
title  
notes  
files[]
```

Response

```
{  
  "intakeId": "",  
  "status": "processing"  
}
```

GET /api/intake/{id}

Response

```
{
  "id": "",
  "status": "",
  "brief": {}
}
```

Brief APIs

GET /api/briefs

Returns employee history.

GET /api/briefs/{id}

Returns full brief.

PATCH /api/briefs/{id}

Request

```
{
  "markdown_output": "",
  "structured_json": {}
}
```

Public Client APIs

GET /api/public/brief/{token}

Returns public brief.

POST /api/public/brief/{token}/answers

Request

```
{
  "responses": [
    {
      "field": "budget",
      "value": "$10,000"
    }
  ]
}
```

POST /api/public/brief/{token}/confirm

Marks brief confirmed.

AI Processing APIs (Internal)

POST /internal/process-intake

Request

```
{
  "intakeId": ""
}
```

9. Frontend Screens

Employee Side

1. Login Page

- Authentication

2. Dashboard

- Recent briefs
- Status tracking
- Missing info alerts

3. New Intake Page

- Drag/drop upload
- Text notes
- Submission state

4. Brief Review Page

- Structured brief viewer/editor
 - Missing fields section
 - Share link copy button
-

Client Side

5. Public Brief View

- Read-only brief
 - Missing info form
 - Confirmation button
-

10. Detailed Task Breakdown

TASK GROUP A — Authentication

Task A1

Description

Implement employee authentication system.

Inputs

Email/password

Outputs

JWT token/session

Owner

Backend

Dependencies

None

TASK GROUP B — Intake Upload System

Task B1

Description

Build frontend upload UI.

Inputs

Files + notes

Outputs

Multipart API request

Owner

Frontend

Dependencies

API endpoint ready

Task B2

Description

Create upload API endpoint and storage logic.

Inputs

Multipart form data

Outputs

Stored files + intake record

Owner

Backend

Dependencies

Database schema

TASK GROUP C — AI Processing

Task C1

Description

Extract text from uploaded files.

Inputs

Images/audio/text

Outputs

Unified extracted text

Owner

Backend AI Service

Dependencies

Upload pipeline

Task C2

Description

Generate structured brief using LLM.

Inputs

Extracted text

Outputs

Structured JSON brief

Owner

Backend AI Service

Dependencies

C1

Task C3

Description

Detect missing information fields.

Inputs

Generated brief

Outputs

Missing field list

Owner

Backend AI Service

Dependencies

C2

TASK GROUP D — Brief Management

Task D1

Description

Store and retrieve structured briefs.

Inputs

Structured brief JSON

Outputs

Persistent brief records

Owner

Backend

Dependencies

Database schema

Task D2

Description

Build employee dashboard and brief history.

Inputs

Brief API data

Outputs

Interactive dashboard UI

Owner

Frontend

Dependencies

D1 APIs

TASK GROUP E — Public Share System

Task E1

Description

Generate secure public share tokens.

Inputs

Brief ID

Outputs

Public URL token

Owner

Backend

Dependencies

D1

Task E2

Description

Build public client review page.

Inputs

Public brief endpoint

Outputs

Read-only review UI

Owner

Frontend

Dependencies

Task E3

Description

Handle client answers and confirmation.

Inputs

Client responses

Outputs

Stored confirmations

Owner

Backend

Dependencies

E2

TASK GROUP F — Brief Editing

Task F1

Description

Allow employee edits to generated briefs.

Inputs

Edited brief data

Outputs

Updated brief

Owner

Dependencies

D1

11. Team Assignment Plan

Khaled — Backend 1

Priority Order

1. Database Schema & Setup

Scope: Medium

2. Authentication APIs

Scope: Medium

3. Intake Upload API

Scope: Large

4. Brief CRUD APIs

Scope: Medium

5. Share Link Generation

Scope: Small

6. Client Confirmation APIs

Scope: Medium

Nady — Backend 2

Priority Order

1. AI Processing Service Architecture

Scope: Medium

2. OCR + Transcription Pipeline

Scope: Large

3. LLM Prompt Engineering

Scope: Large

4. Missing Information Detection

Scope: Medium

5. Internal AI Processing APIs

Scope: Medium

Gom3a — Frontend

Priority Order

1. App Routing + Layout

Scope: Medium

2. Login Page

Scope: Small

3. Upload Intake UI

Scope: Large

4. Dashboard + History

Scope: Medium

5. Brief Review UI

Scope: Large

6. Public Client View

Scope: Medium

12. Parallel Work Strategy

Can Start Immediately in Parallel

Khaled

- DB schema
- Auth APIs
- Intake APIs

Nady

- AI service prototype
- Prompt engineering
- OCR pipeline

Gom3a

- Frontend layout
- Upload UI mock
- Dashboard UI

Sequential Dependencies

AI Processing Depends On

- Upload storage working

Dashboard Depends On

- Brief APIs ready

Public Share View Depends On

- Share token APIs ready

13. Validation Strategy

Frontend Validation

- File type checks
- Required field checks
- Input formatting

Backend Validation

- Token validation
- File size/type enforcement
- Schema validation
- Authorization

AI Validation

- Ensure JSON structure consistency
 - Reject malformed outputs
-

14. Error Handling

Upload Errors

```
{  
  "error": "Unsupported file type"  
}
```

AI Failures

```
{  
  "error": "AI processing failed",  
  "retryable": true  
}
```

Public Link Errors

```
{  
  "error": "Invalid or expired link"  
}
```

15. Recommended Sprint Timeline

Day 1

- Database
- Backend skeleton
- Frontend skeleton

Day 2

- Upload flow
- File storage
- Dashboard UI

Day 3

- AI extraction pipeline
- LLM generation

Day 4

- Brief rendering
- Shareable links

Day 5

- Client confirmation flow
- Missing info prompts

Day 6

- Integration testing
- UI refinement

Day 7

- Demo prep
 - Bug fixing
 - Deployment
-

16. MVP Success Criteria

The submission is successful if:

1. Employee uploads messy input
2. AI generates structured brief
3. Missing info is detected
4. Client opens shareable link
5. Client confirms or fills missing info
6. Employee sees updated confirmed brief
7. All briefs persist in database history